Veritabanı Performans Optimizasyonu ve İzleme

Proje - 1

Semih Cengiz - 20290233

BLM4522 - Ağ Tabanlı Paralel Dağıtım Sistemleri

Giriş

Bu projede örnek veritabanı kullanarak PostgreSQL ile performans analizi ve optimizasyon çalışmaları yapılmıştır. Çalışmanın amacı; sorgu performansını izleme, mevcut indeks yapısını kontrol etme, uzun süren sorguları analiz etme gibi konular içermektedir. Ayrıca disk alanı kullanımı ve veri yöneticisi rolleri gibi başlıklar incelenmiştir.

Kullanılan Veritabanı

Bu çalışmada PostgreSQL için özel olarak hazırlanmış Pagila veritabanı kullanılmıştır. Pagila DVD kiralama sistemine benzer bir yapıya ve tablolar sahiptir. Örneğin customer, film, mental ,payment gibi tablolar içermektedir. Projede mevcut yapı incelenmiş ve performans testleri için orijinal veritabanını bozmamak için ayrı bir çalışma veritabanı oluşturulmuştur.

Başlangıç Durumunun Analizi

table_name	row_count
film	1000
actor	200
customer	599
inventory	4581
rental	16044
payment	16044

Tabloların Satır Sayısı

Yukarıda gösterilen tablo veritabanının başlangıçta içerdiği tabloların büyüklüğünü göstermektedir. En yoğun tabloların rental ve payment olduğu görülüyor. Ancak proje konuları dokümanında büyük bir veritabanı olarak bahsedildiği için ilerleyen aşamalarda daha büyük bir veritabanı oluşturulacaktır.

schemaname	table_name	total_size	total_size_bytes
public	rental	1920 kB	1966080
public	film	752 kB	770048
public	payment_p2007_03	512 kB	524288
public	film_actor	496 kB	507904
public	inventory	464 kB	475136
public	payment_p2007_04	440 kB	450560
public	payment_p2007_02	408 kB	417792

En Çok Yer Kaplayan Tablolar

Yukarıda tabloda tabloların diskte ne kadar yer kapladığı görülmektedir. Burada en fazla yer kaplayan rental tablosu olduğundan performans testlerinde özellikle kiralama işlemleriyle ilgili tabloların dikkate alınması gerektiğini göstermektedir. Ayrıca payment tablolarının parçalanmış halde bulunduğu görülmektedir.

schemaname	tablename	indexname	indexdef
public	actor	actor_pkey_incl	CREATE UNIQUE INDEX actor_pkey_incl ON public.actor USING btree (actor_id) INCLUDE (first_name, last_name)
public	actor	idx_actor_last_name	CREATE INDEX idx_actor_last_name ON public.actor USING btree (last_name)
public	film	film_fulltext_idx	CREATE INDEX film_fulltext_idx ON public.film USING gist (fulltext)
public	film	film_pkey	CREATE UNIQUE INDEX film_pkey ON public.film USING btree (film_id)
public	film	idx_fk_language_id	CREATE INDEX idx_fk_language_id ON public.film USING btree (language_id)
public	film	idx_fk_original_language_id	CREATE INDEX idx_fk_original_language_id ON public.film USING btree (original_language_id)
public	film	idx_title	CREATE INDEX idx_title ON public.film USING btree (title)
public	customer	customer_pkey	CREATE UNIQUE INDEX customer_pkey ON public.customer USING btree (customer_id)
public	customer	idx_fk_address_id	CREATE INDEX idx_fk_address_id ON public.customer USING btree (address_id)
public	customer	idx_fk_store_id	CREATE INDEX idx_fk_store_id ON public.customer USING btree (store_id)
public	customer	idx_last_name	CREATE INDEX idx_last_name ON public.customer USING btree (last_name)

Mevcut İndekslerin İncelenmesi

Başlangıç aşamasında zaten birçok indeksin tanımlı olduğu görülmektedir. Özellikle primary key ve foreign key alanlarında indeksler bulunmaktadır. Ayrıca film.title, actor.last_name ve customer.last_name gibi sorgularda sık kullanılabilecek alanlar için indeks tanımlamaları mevcuttur. Bu nedenle öncelikle optimizasyon için doğrudan indeksleri silmek yerine, önce hangi indekslerin gerçekten etkili olduğunun ölçülmesi planlanmaktadır.

Veritabanı izleme

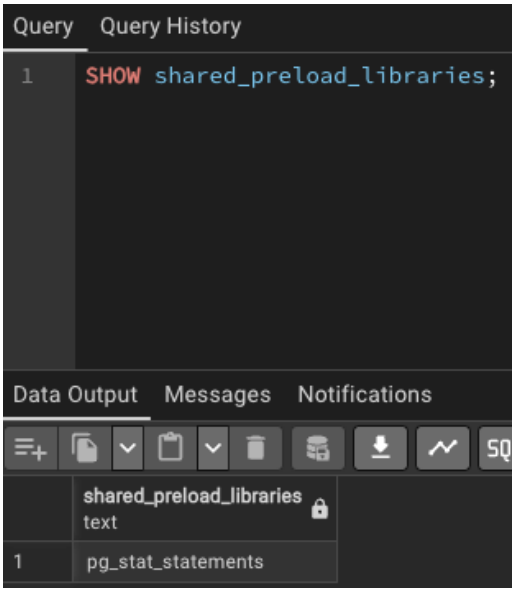
Bu bölümde pagila veritabanı için PostgreSQL'de bulunan **pg_stat_activity**, **pg_stat_statements** ve **log ayarları** modülü incelenmiştir. Bu araçlar sayesinde aktif sorgular, uzun süren işlemler ve performans problemi oluşturabilecek sorgular gözlemlenebilmektedir.

Modül Ayarları

Modüllerin çalışabilmesi için config dosyasında bazı değişiklikler yapmak gerekiyordu. postgresql.conf içindeki:

shared_preload_libraries = 'pg_stat_statements',

satırı varsayılan olarak boştu, onu pg_stat_statements ile değiştirdim. Bu adım, sorguların istatistiksel olarak izlenebilmesi için gereklidir.



SHOW Sorgusu İle Teyit

Eklenti Oluşturma

Yapılandırma ayarları tamamlandıktan sonra Pagila veritabanısında **pg_stat_statements** eklentisi oluşturulması gerekmektedir. Bu eklenti sayesinde veritabanısında çalışan sorguların çağrı sayısı, toplam çalışma süresi ve ortalama çalışma süresi gibi istatistikler elde edilebilmektedir.

```
CREATE EXTENSION IF NOT EXISTS pg_stat_statements;
SELECT * FROM pg_stat_statements LIMIT 5;
```

Sorgular

calls bigint	total_exec_time double precision	min_exec_time double precision	max_exec_time double precision	mean_exec_time double precision	stddev_exec_time double precision
5	0.44625	0.052375	0.138041	0.08925	0.035559982975249024
7	0.15487499999999998	0.011875	0.0375	0.022125000000000002	0.008901610127226582
7	6.663416	0.479125	1.4835	0.9519165714285714	0.4047192750236558
2	0.03229	0.007541	0.024749	0.016145	0.008604
5	0.029042	0.002666	0.00725	0.0058084	0.0016268111875691047

pg_stat_statements Sorgusundan Bir Parça

Yukarıdaki ekran görüntüsünde sorguların kaç defa çalıştığı, çalışma süreleri vs. görülebilmektedir.

Aktif Sorguları İzleme

PostgreSQL'de aktif oturumları ve çalışan sorguları izlemek için **pg_stat_activity** görünümü kullanılmıştır. Bu görünüm, o anda çalışan işlemleri, kullanıcı bilgisini, sorgu başlama zamanını ve bekleme durumlarını göstermektedir. Test amacıyla bir oturumda SELECT pg_sleep(20); sorgusu çalıştırılmış ve diğer oturumda **pg_stat_activity** ile bu sorgunun aktif olarak sistemde göründüğü gösterilmiştir. Böylece anlık sorgu izleme işlemi gerçekleştirilmiştir.

Query Query History

```
    usename,
    datname,
    state,
    wait_event_type,
    wait_event,
    now() - query_start AS duration,
    query
FROM pg_stat_activity
WHERE state <> 'idle'
ORDER BY duration DESC;
```

Data Output Messages Notifications

Showing rows: 1 to 2 Page No: 1 of 1

	pid integer	usename name	datname name	state text	wait_event_type text	wait_event text	duration interval	query text
1	27401	macbook	pagila	active	Timeout	PgSleep	00:00:08.6264...	SELECT pg_sleep(20);
2	27388	macbook	pagila	active	[null]	[null]	00:00:00	SELECT pid, usename, datname, stat

Aktif Sorguları Görme

Yukarıdaki tabloda diğer oturumda aktif olarak çalışan sleep sorgusunu, ne kadar zamandır çalıştığını vs. görebiliyoruz.

En Çok Zaman Harcayan Sorguları Tespit Etme

```
SELECT
    calls,
    round(total_exec_time::numeric, 2) AS total_ms,
    round(mean_exec_time::numeric, 2) AS mean_ms,
    rows,
    query
FROM pg_stat_statements
ORDER BY mean_exec_time DESC
LIMIT 10;
```

Ortalama Süreye Göre Sıralama

pg_stat_statements, sorguların çağrı sayısını, toplam planlama ve yürütme süresini, ortalama süreyi ve satır sayılarını gösterir.

	calls bigint	total_ms numeric	mean_ms numeric	rows bigint	query text
1	3	50003.45	16667.82	3	SELECT pg_sleep($1)
2	1	22.05	22.05	0	CREATE EXTENSION IF NOT E
3	3	40.68	13.56	2874	SELECT f.title, COUNT(r.r
4	3	36.17	12.06	1797	SELECT c.customer_id, c.
5	1	2.51	2.51	0	ALTER SYSTEM SET log_min_
6	2	4.44	2.22	0	ALTER SYSTEM SET deadlock
7	1	1.50	1.50	0	ALTER SYSTEM SET log_lock_
8	5	7.24	1.45	5	SELECT set_config($1,$2,$3)
9	19	18.23	0.96	442	SELECT DISTINCT att.attname
10	1	0.87	0.87	1	SELECT set_config($1,$2,$3)

Sorguların Çalışma Zamanına Göre Sıralanmış Hali

Yavaş Sorguları Loglama

Yavaş süren sorguları kayıt altına almak için **log_min_duration_statement** parametresi kullanılmıştır. Test amaçlı bu değer 0 olarak belirlenmiştir ki tüm sorguları görebilelim fakat isteğe bağlı bu değer değiştirilerek beklenenden fazla uzun süren sorgular loglanabilir, tespiti halinde sorunun nerede olduğu görülebilir.

```
2026-04-05 15:01:26.441 +03 [28542] LOG:  starting PostgreSQL 17.6 (Homebrew) on aarch64-apple-darwin24.4.0, compiled by Apple
clang version 17.0.0 (clang-1700.0.13.3), 64-bit
2026-04-05 15:01:26.442 +03 [28542] LOG:  listening on IPv6 address "::1", port 5432
2026-04-05 15:01:26.442 +03 [28542] LOG:  listening on IPv4 address "127.0.0.1", port 5432
2026-04-05 15:01:26.442 +03 [28542] LOG:  listening on Unix socket "/tmp/.s.PGSQL.5432"
2026-04-05 15:01:26.445 +03 [28546] LOG:  database system was shut down at 2026-04-05 15:01:25 +03
2026-04-05 15:01:26.449 +03 [28542] LOG:  database system is ready to accept connections
2026-04-05 15:01:33.620 +03 [28574] LOG:  duration: 4.047 ms  statement: SET DateStyle=ISO; SET client_min_messages=notice;
SELECT set_config('bytea_output','hex',false) FROM pg_show_all_settings() WHERE name = 'bytea_output'; SET
client_encoding='utf-8';
2026-04-05 15:01:33.625 +03 [28574] LOG:  duration: 0.296 ms  statement: SELECT version()
2026-04-05 15:01:33.627 +03 [28574] LOG:  duration: 2.535 ms  statement:
    SELECT
        db.oid as did, db.datname, db.datallowconn,
        pg_encoding_to_char(db.encoding) AS serverencoding,
        has_database_privilege(db.oid, 'CREATE') as cancreate,
        datistemplate
```

Log Dosyasının Bir Kısmı

Yukarıda PostgreSQL servisinin başlatıldığını, Sunucunun bağlantıyı kabul etmeye hazılandığını, her bir işlemin ne kadar sürdüğü görülebilmektedir. Burada minimum Log zamanı 0 yerine başka bir değer ayarlanırsa, filtreleme ile istenilen sonuç daha rahat görülebilir.